JAVA

Sudhir Lama

TABLE OF CONTENTS

01 Basics of Programming in Java

02 Object Oriented Programming in Java

03 Building Components using Swing

04 Distributed Networking Programming

05 Database Connectivity

06 Servlets and JSP

07 Advanced Topics in Java

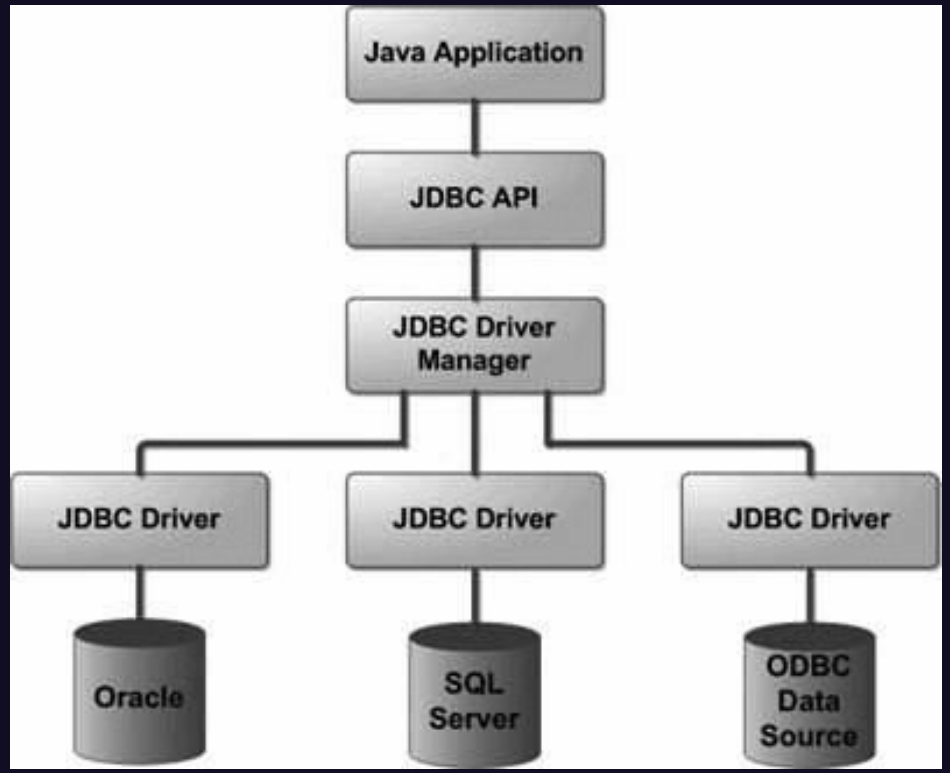
Database Connectivity

5

WHAT IS JDBC?

- JDBC (Java Database Connectivity) is a Java-based API (Application Programming Interface) that serves as a standardized framework for enabling Java applications to interact with relational databases.
- It facilitates the establishment of database connections, execution of SQL queries, retrieval of data, and modification of database records.
- It acts as a bridge between a Java application and a database, enabling seamless communication and data exchange. It is widely used in Java applications for integrating database functionality.
- It is an important tool for creating Java applications that need to work with databases as it makes easier to connect to and interact with databases, providing a common way to work with different database systems.
- Whether you're building web apps, desktop programs, or server-side applications, JDBC helps you easily add database features to your Java projects
- It is a API that is used to interact with databases, issue queries and commands and process database result sets. JDBC and database drivers operate together to access spreadsheets and databases. The API classes and interfaces of JDBC allow an application to send a request to a particular database. It helps Java applications interact with ORACLE, MYSQL, and MSSQL databases.

JDBC Architecture



JDBC Architecture

- Java Application
  - In the JDBC Architecture, the Java application serves as the foundation and primary layer that interfaces with the JDBC API. It is where developers implement the business logic to establish database connections, execute queries, and handle the resulting data.
- JDBC Api
  - The JDBC API (Java Database Connectivity Application Programming Interface) is a part of Java's standard library that provides a structured way for Java applications to interact with relational databases. It includes a set of classes, interfaces, and methods that enable developers to establish database connections, execute SQL queries, retrieve results and update database records.
- JDBC Manager Driver
  - The JDBC DriverManager is a core component of Java Database Connectivity (JDBC) that helps manage a set of database drivers. It acts as an interface between Java applications and the various database management systems (DBMS). The DriverManager class is responsible for managing database drivers and establishing a connection to the database.
  - It acts as a connection factory for creating database connections and provides the mechanism to establish a link between a Java application and a specific database using an appropriate JDBC driver.

JDBC Architecture

- JDBC Driver
  - These drivers are used to establish connections with various data sources. Different databases like MSSQL, Oracle, and MySQL each have their own specific drivers. To connect with these databases, we need to load their respective drivers. The Java class used for loading these drivers is Class, and the Class.forName() method is employed to load the drivers in the JDBC architecture.
- Data Source
  - These are the databases that can be accessed using the JDBC API. They serve as the storage sources for data, which can be utilized by Java applications. The JDBC API facilitates connections to various databases, including MySQL, MSSQL, PostgreSQL, Oracle, and others.

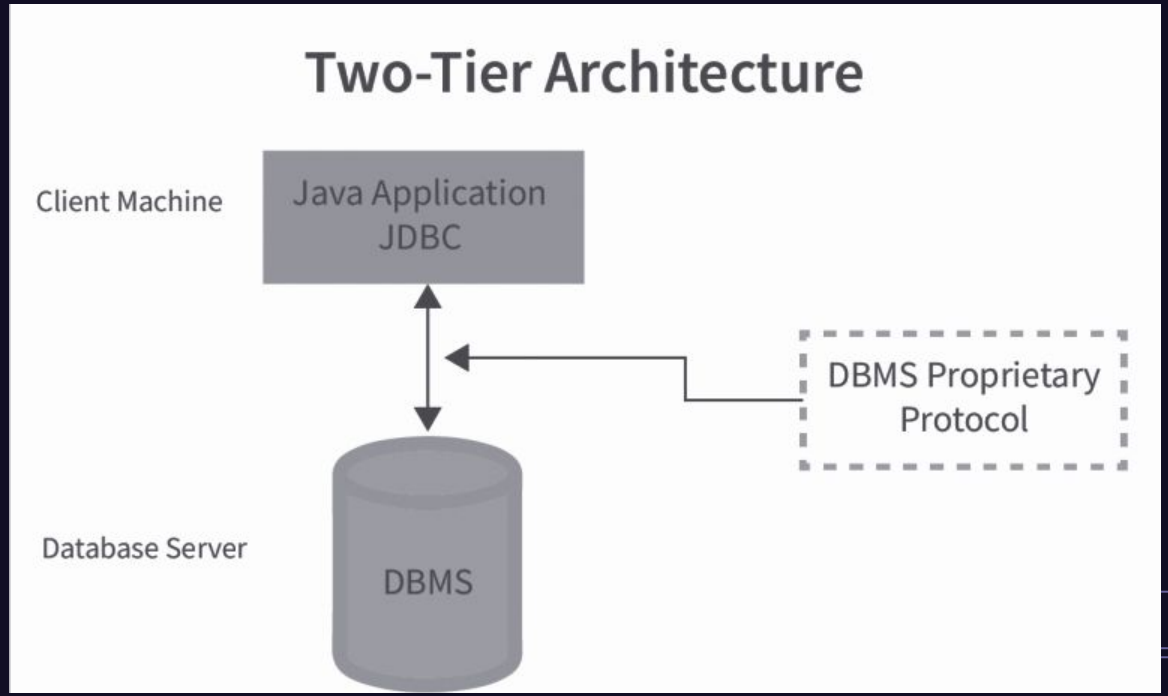
JDBC Architecture types

Two-Tier Architecture

- It is a basic model, where a Java application and Java applet (Client) communicate directly with the data source.
- JDBC acts as the bridge between the client application and the database.
- This architecture is typically used in small applications where the database and application are on the same or a closely connected system.
- The user's machine acts as a client, and the machine has the data source running acts as the server.
- Direct interaction between the client and database which makes it simple.
- Faster communication due to fewer layers.
- Limited scalability as all clients must maintain a connection to the database.

JDBC Architecture types

- Two-Tier Architecture



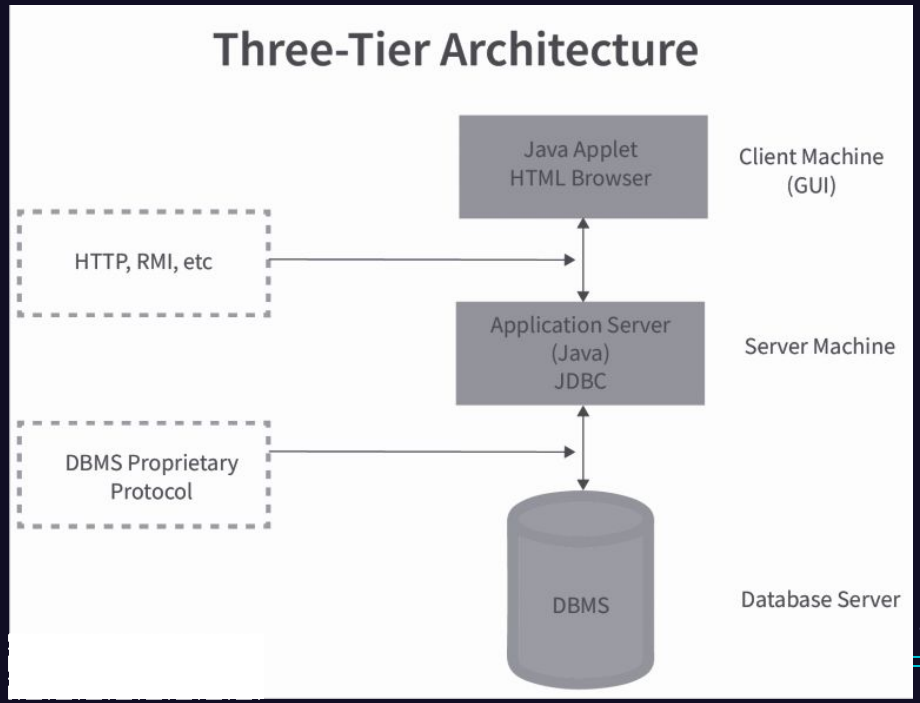
JDBC Architecture types

Three-Tier Architecture

- This Model is a more secure and complex model of JDBC architecture in Java The user queries are sent to the middleware tier and then executed on the data source
- The results received on the database are again sent to the middle tier and then to the application.
- There is an intermediate layer (often called the middleware) between the client and the database which can include an application server, web server, or any other logic-processing entity.
- Is more secure as client does not directly interact with the database.
- Is more scalable as middleware can manage connections to the database, allowing more clients to be supported.
- Business logic is separated into the middleware, which makes the system more modular and flexible.

JDBC Architecture types

- Three-Tier Architecture



JDBC Drivers

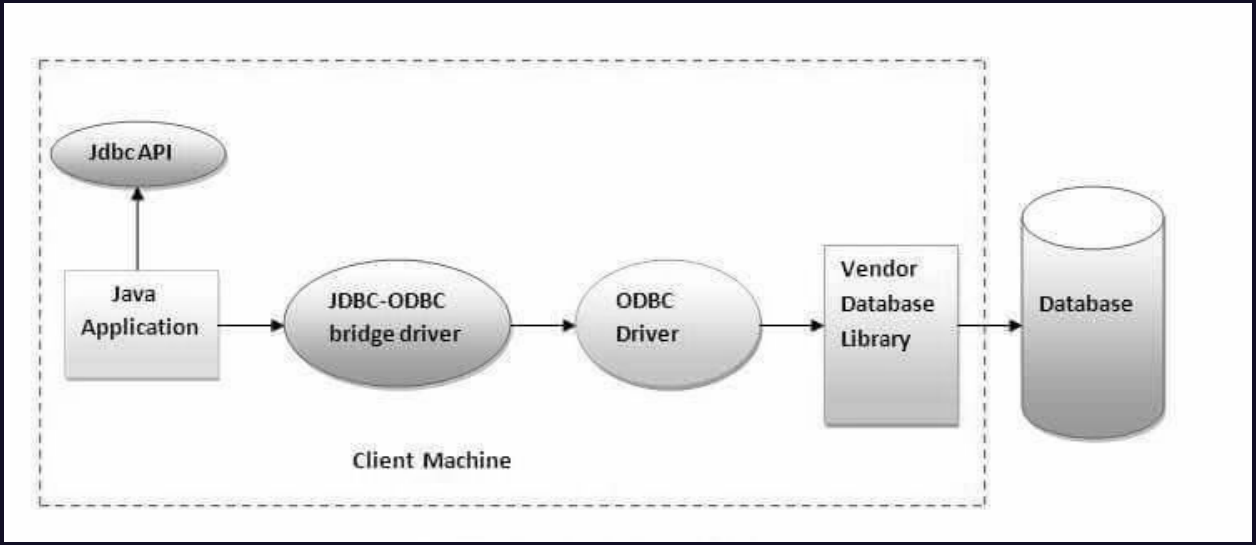
A JDBC driver is a software tool that helps Java applications communicate with databases. It works like a translator, converting Java instructions into commands that the database can understand. Using the JDBC driver, applications can send queries to the database, retrieve results, and update data in various types of databases. It handles the communication with the database server.

Type 1 : JDBC-ODBC bridge driver[Bridge Driver]

- The JDBC-ODBC Bridge Driver uses ODBC (Open Database Connectivity) to connect Java applications to a database. It translates JDBC calls into ODBC calls, which are then handled by the ODBC driver manager.
- The JDBC-ODBC bridge driver uses ODBC driver to connect to the database.
- The ODBC driver is platform-dependent, which makes this driver less efficient and deprecated in later versions of JDBC.
- Easy to use and can be easily connect to any database.
- As a drawback, ODBC is platform-specific, so the application needs to be on the same platform as the database.
- Performance bottleneck due to the translation layer between JDBC and ODBC.
- The ODBC driver needs to be installed on the client machine.

▶ JDBC Drivers

Type 1 : JDBC-ODBC bridge driver[Bridge Driver]



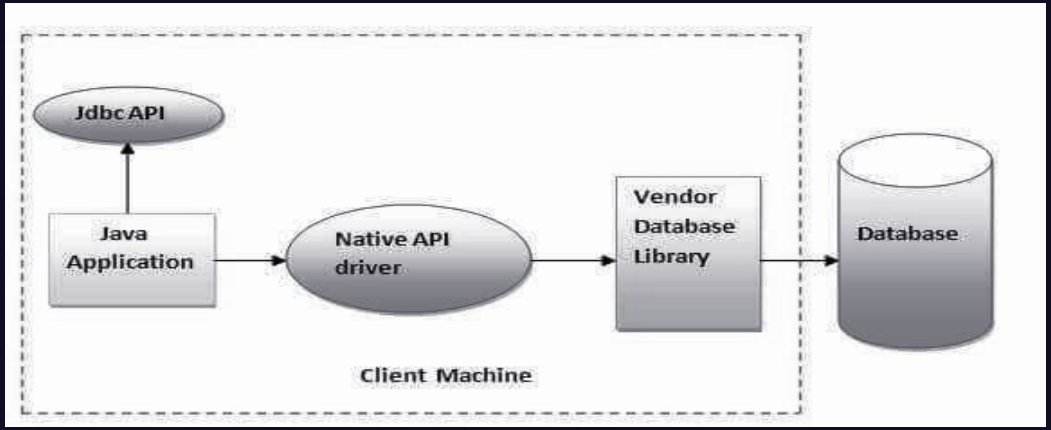
JDBC Drivers

Type 2 : Native API Driver[Partially Java Driver]

- The Native API driver relies on the database's client-side libraries for communication.
- It translates JDBC method calls into the native API calls specific to the database and to work with different databases, this driver requires the database's local API.
- This approach offers better security for data transfer compared to the Type-1 driver.
- The Native-API driver converts JDBC calls into database-specific native calls (using libraries written in languages like C, C++, or other low-level languages). These native calls are then used to communicate with the database.
- This driver communicates with the database using a native database client (for example, a native MySQL client or Oracle client) installed on the local machine.
- Performance upgraded than JDBC-ODBC bridge driver
- As a drawback, driver needs to be installed separately in individual client machines
- The Vendor client library needs to be installed on client machine.
- It is a database dependent driver.

▶ JDBC Drivers

Type 2 : Native API Driver[Partially Java Driver]



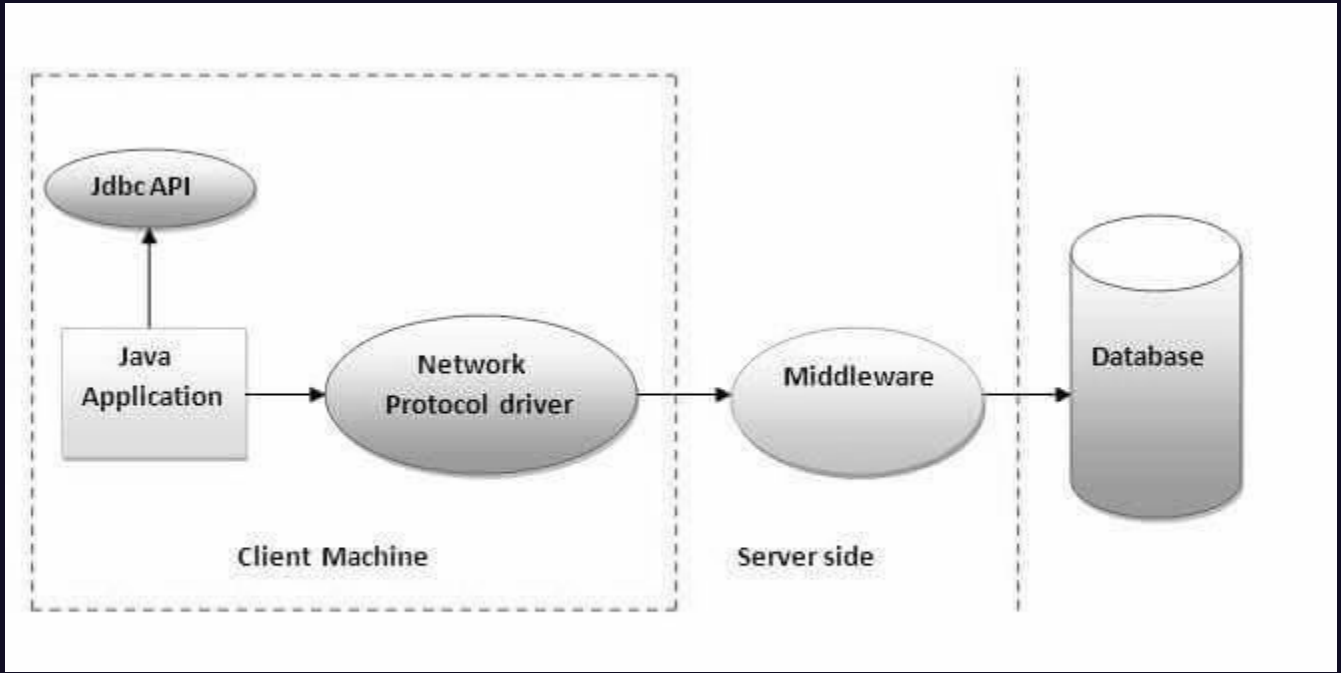
JDBC Drivers

Type 3 : Network Protocol Driver[Middleware Driver]

- The Network Protocol Driver communicates with the database server using a middle layer also known as middleware server which translates the JDBC calls into a database-specific protocol understood by the database.
- The driver sends the database communication over a network. When a JDBC call is made, it is passed to the middleware server, which then converts the call into the database-specific protocol and communicates with the database.
- It is platform independent as The driver can work across different platforms because the communication is done through a middle layer.
- As a drawback, it slower due to the added layer (middleware) and network latency.
- Requires additional infrastructure for the middleware and network support is required in client's machine.
- Maintenance of Network Protocol driver becomes costly because it requires database-specific coding to be done in the middle tier.

▶ JDBC Drivers

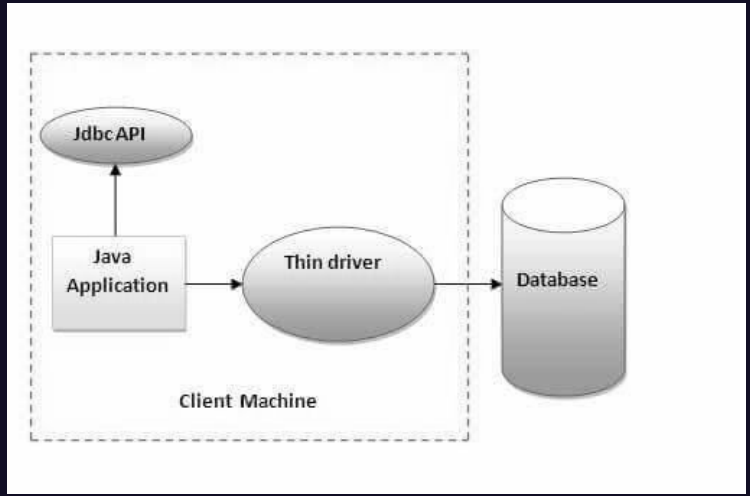
Type 3 : Network Protocol Driver[Middleware Driver]



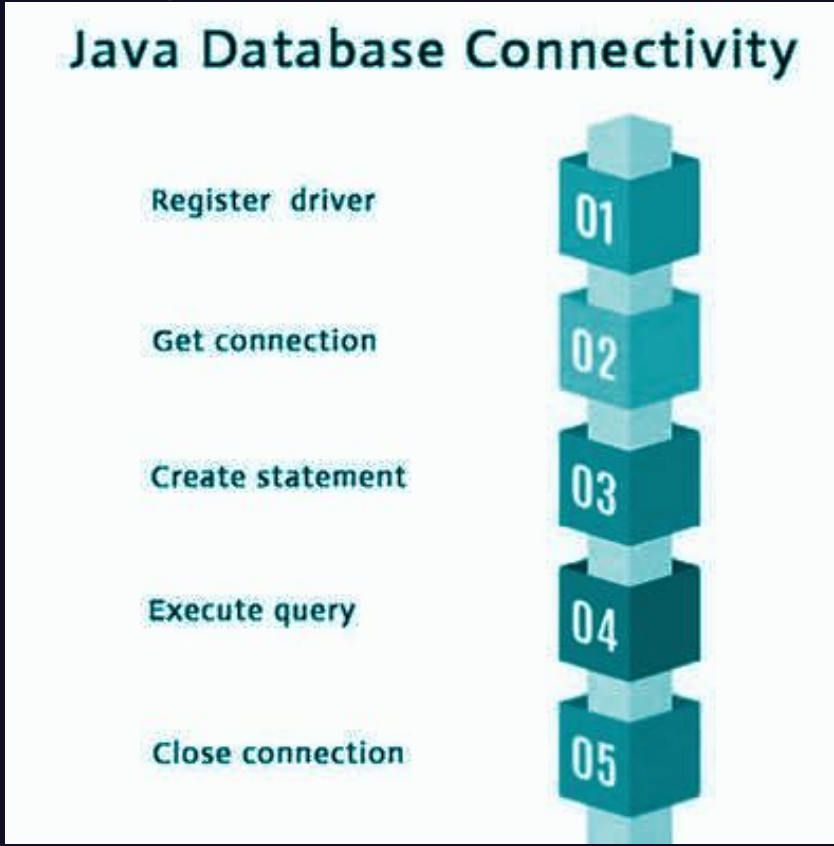
JDBC Drivers

Type 4 : Thin Driver[Pure Java Driver]

- Thin driver is a pure Java driver that communicates directly with the database using its native protocol, without requiring any native library or middleware.
- It's the most commonly used JDBC driver in modern Java applications because of its portability and efficiency.
- No software is required on either server or client side.
- Direct connection to the database without intermediate translation layers so it is better performance than all the drivers.



▶ JDBC Connectivity



▶ JDBC Connectivity

❖ Register Drivers

- ➢ The Class.forName() method is used to register the driver class. This method is used to dynamically load the driver class.
- ➢ Modern JDBC drivers like MySQL may no longer require this step as drivers are auto-registered since JDBC 4.0.

```
Class.forName("com.mysql.cj.jdbc.Driver");
```

▶ JDBC Connectivity

❖ Create the connections

➢ The getConnection() method of DriverManager class is used to establish connection with the database.

```
Connection connection = DriverManager.getConnection(URL, username, password);
```

- *URL*
  - *The databaseURL typically in the format*
    - *jdbc:mysql://hostname:port/databaseName*
- *username*
  - *Database username*
- *password*
  - *Database password*

Eg:

```
String url = "jdbc:mysql://localhost:3306/javadatabase";
String username = "root";
String password = "password";
Connection connection = DriverManager.getConnection(url, username, password);
```

▶ JDBC Connectivity

❖ Create a Statement

- ➢ The Statement object is responsible for executing queries with the database.
- ➢ This object lets to execute SQL queries like SELECT, INSERT, UPDATE and DELETE.

```
Connection connection = DriverManager.getConnection(URL, username, password);
Statement statement = connection.createStatement();
```

▶ JDBC Connectivity

❖ Execute Query

- The executeQuery() method of Statement interface is used to execute queries to the database. This method returns the object of ResultSet that can be used to get all the records of a table.
  - executeQuery: Used for retrieving data (SELECT statements). Returns a ResultSet.
  - executeUpdate: Used for updating data (INSERT, UPDATE, DELETE). Returns an integer representing the number of rows affected.

```
ResultSet resultSet = statement.executeQuery("SELECT * FROM student");
```

▶ JDBC Connectivity

❖ Close the Connection

- ➢ This ensures the application doesn't leak memory or connections.

```
resultset.close()
statement.close()
connection.close()
```

```java
import java.sql.*;

public class JDBCExample {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/javadatabase";
        String user = "ktmncit";
        String password = "ncIT#123";
        try {
            // Load the driver (optional for JDBC 4.0+)
            Class.forName("com.mysql.cj.jdbc.Driver");
            // Establish the connection
            Connection connection = DriverManager.getConnection(url, user, password);
            System.out.println("Connected to the database!");
            // Create a statement
            Statement statement = connection.createStatement();
            // Execute a query; before to execute we need to have student table in our javadatabase
            String query = "SELECT * FROM student";
            ResultSet resultSet = statement.executeQuery(query);
            // Process the results
            while (resultSet.next()) {
                int id = resultSet.getInt("id");
                String name = resultSet.getString("name");
                System.out.println("ID: " + id + ", Name: " + name);
            }
            // Close the resources
            resultSet.close();
            statement.close();
            connection.close();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

▶ Result Set

- In JDBC , a ResultSet is an object that holds the data retrieved from a database after executing a SQL query. It acts as a pointer to the rows of the query result. The ResultSet object allows iteration through the retrieved rows and access to column values.
- The ResultSet object contains a cursor that points to the current row of data. Initially, the cursor is positioned before the first row, and it must be moved to access the rows.
- It is essentially a table of data representing the result of a database query.
- Based on cursor movement and operations, by default Result set is forward only and it read only respectively.
- For retrieving Data: The data can be retrieved from a ResultSet using methods like getString(), getInt(), getDate() etc.
- To Navigate: we can move through the rows of the result set, typically starting from the first row, using methods like next(), previous(), first() and last().

▶ Scrollable Result Set

- A scrollable ResultSet provides more flexibility by allowing both forward and backward movement through the data, which can be very useful in various scenarios like searching, updating or viewing large datasets.
- It is controlled when creating the statement.

```
Statement stmt = connection.createStatement(ResultSet.TYPE_SCROLL_INSENSITIVE,
ResultSet.CONCUR_READ_ONLY);
ResultSet rs = stmt.executeQuery("SELECT * FROM students");
```

▶ Updatable Result Set

- An updatable ResultSet in JDBC enables efficient data modification and management directly in the ResultSet, offering a convenient way to handle database changes in Java applications.
- It allows to retrieve data from a database and modify it directly within the ResultSet, which can then be reflected back in the database.

```
Statement stmt = conn.createStatement(ResultSet.TYPE_SCROLL_INSENSITIVE,
ResultSet.CONCUR_UPDATABLE);
ResultSet rs = stmt.executeQuery("SELECT * FROM students");
```

Scrollable and Readonly

```
import java.sql.*;

public class ScrollableResultSetExample {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/javadatabase";
        String user = "ktmncit";
        String password = "ncIT#123";
        try (Connection conn = DriverManager.getConnection(url, user, password);
            Statement stmt = conn.createStatement(
                    ResultSet.TYPE_SCROLL_INSENSITIVE,
                    ResultSet.CONCUR_READ_ONLY
            );
            ResultSet rs = stmt.executeQuery("SELECT id, name, age FROM students")) {

            // Move to the last row and display data
            if (rs.last()) {
                System.out.println("Last Row - ID: " + rs.getInt("id") +
                                   ", Name: " + rs.getString("name") +
                                   ", Age: " + rs.getInt("age"));
            }
            // Move to the first row and display data
            if (rs.first()) {
                System.out.println("First Row - ID: " + rs.getInt("id") +
                                   ", Name: " + rs.getString("name") +
                                   ", Age: " + rs.getInt("age"));
            }
            // Move to the second row and display data
            if (rs.absolute(2)) {
                System.out.println("Second Row - ID: " + rs.getInt("id") +
                                   ", Name: " + rs.getString("name") +
                                   ", Age: " + rs.getInt("age"));
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

Scrollable and Updatable

```
import java.sql.*;
public class UpdatableResultSetExample {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/javadatabase";
         String user = "ktmncit";
        String password = "ncIT#123";
        try (Connection conn = DriverManager.getConnection(url, user, password);
             Statement stmt = conn.createStatement(
                    ResultSet.TYPE_SCROLL_SENSITIVE,
                    ResultSet.CONCUR_UPDATABLE
             );
             ResultSet rs = stmt.executeQuery("SELECT id, name, age FROM students")) {
            // Update the first row
            if (rs.first()) {
                rs.updateString("name", "Updated Name");
                rs.updateInt("age", 35);
                rs.updateRow();
                System.out.println("Row updated.");
            }
            // Insert a new row
            rs.moveToInsertRow();
            rs.updateInt("id", 104);
            rs.updateString("name", "New User");
            rs.updateInt("age", 28);
            rs.insertRow();
            System.out.println("New row inserted.");
            // Delete the last row
            if (rs.last()) {
                rs.deleteRow();
                System.out.println("Last row deleted.");
            }

        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

▶ Exception Handling in JDBC

Exception Handling is a mechanism in programming used to manage errors or unexpected events during program execution. It ensures that the program continues to run smoothly or terminates gracefully without crashing, while addressing the issue that caused the error.

- Importance of Exceptional handling in JDBC
  - Ensures application stability when database errors occur.
  - Helps prevent data corruption (e.g., incomplete transactions).
  - Provides meaningful error messages for developers to debug issues.
  - Automatically cleans up database resources like connections, avoiding memory leaks.
  - Enhances the user experience by providing user-friendly error messages.

▶ Common Exception in JDBC

- SQL Exception
  - Generic exception for SQL-related issues (e.g., syntax errors, connection failure).
- SQLIntegrityConstraintViolationException:
  - Occurs when constraints like primary key, foreign key, or unique constraints are violated. Like trying to insert duplicate primary key
- SQLSyntaxErrorException
  - Triggered by invalid SQL syntax. Like spelling mistake in writing SELECT
- SQLTransientException
  - Indicates temporary issues, such as network glitches or timeouts.
- SQLNonTransientException:
  - Indicates permanent issues, such as using a non-existent table.

▶ Basic Exception Handling in JDBC

```
try {
    // Database operations
} catch (SQLException e) {
    System.err.println("Error: " + e.getMessage());
    System.err.println("SQL State: " + e.getSQLState());
    System.err.println("Error Code: " + e.getErrorCode());
}
```

Specific Exception Handling in JDBC

```
try {
    // Database operations
} catch (SQLIntegrityConstraintViolationException e) {
    System.err.println("Constraint violation: " + e.getMessage());
} catch (SQLSyntaxErrorException e) {
    System.err.println("Syntax error: " + e.getMessage());
} catch (SQLException e) {
    System.err.println("General database error: " + e.getMessage());
}
```

Transaction Management with Exception Handling

```
try (Connection conn = DriverManager.getConnection("jdbc:mysql://localhost:3306/mydb", "user", "password")) {
    conn.setAutoCommit(false); // Start transaction
    Statement stmt = conn.createStatement();
    stmt.executeUpdate("INSERT INTO users (name) VALUES ('John')");
    stmt.executeUpdate("INSERT INTO orders (user_id, amount) VALUES (LAST_INSERT_ID(), 100)");
    conn.commit(); // Commit transaction
} catch (SQLException e) {
    if (conn != null) {
        try {
            conn.rollback(); // Rollback on error
        } catch (SQLException rollbackEx) {
            System.err.println("Rollback failed: " + rollbackEx.getMessage());
        }
    }
    System.err.println("Transaction failed: " + e.getMessage());
}
```

▶ DDL

DDL commands are used to define, modify, or delete the structure of database objects like tables, schemas, indexes, or constraints. These commands change the database schema.

- CREATE
  - Creates a new database, table, index or other database objects.

```
//Creating a database
CREATE DATABASE my_database;


// Creating a table
CREATE TABLE employees (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100),
    department VARCHAR(50),
    salary DECIMAL(10, 2)
);
```

DDL

- ALTER
  - Modifies the structure of an existing table or database object.

```
// Adding a new column to an existing table

ALTER TABLE employees ADD COLUMN hire_date DATE;
```

DDL

- DROP
  - Deletes an existing database, table or database object.

```
//Deleting a table
DROP TABLE employees;

//Deleting a database
DROP DATABASE my_database;
```

DDL

- TRUNCATE
  - Removes all rows from a table without logging individual row deletions.

```
//Removing all data from a table:

TRUNCATE TABLE employees;
```

DDL Example

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.Statement;

public class DDLExample {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/mydatabase";
        String user = "root";
        String password = "password";
        try (Connection connection = DriverManager.getConnection(url, user, password);
             Statement statement = connection.createStatement()) {
            // Create a table
            String createTableSQL = "CREATE TABLE users ("
                                    + "id INT PRIMARY KEY AUTO_INCREMENT, "
                                    + "name VARCHAR(50), "
                                    + "email VARCHAR(50))";
            statement.execute(createTableSQL);
            System.out.println("Table created successfully!");
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

▶ DML

DML commands are used to manipulate the data within tables. They are concerned with adding, updating, deleting, or retrieving data in a database

- **INSERT**
  - Adds new rows (records) into a table.

```
// Adding a new row into a employees table
INSERT INTO employees (name, department, salary)
VALUES (Ram, 'HR', 40000);

//Adding multiple rows
INSERT INTO employees (name, department, salary)
VALUES
(‘Shyam’, 'IT', 90000),
(‘Sita’, 'Finance', 55000);
```

DML

UPDATE

- Modifies existing data in a table.

```
// Updating the salary of a specific employee

UPDATE employees
SET salary = 30000
WHERE name = 'Ram';
```

DML

DELETE

- Removes specific rows from a table.

```
//Removing a specific row

DELETE FROM employees
WHERE name = 'Shyam';

//Removing all rows from a table

DELETE FROM employees;
```

▶ DML

❖ SELECT

➢ Retrieves data from one or more tables.

```
//Retrieving all columns
 SELECT * FROM employees;

//Retrieving specific columns
SELECT name, salary FROM employees;

//Filtering rows with conditions
SELECT * FROM employees
WHERE salary > 50000;

//Sorting Results
SELECT * FROM employees
ORDER BY salary DESC;
```

DML Example

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.Statement;
public class DMLExample {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/mydatabase";
        String user = "root";
        String password = "password";

        try (Connection connection = DriverManager.getConnection(url, user, password);
             Statement statement = connection.createStatement()) {
            // Insert data
            String insertSQL = "INSERT INTO users (name, email) VALUES ('Ram', 'ram@example.com')";
            int rowsInserted = statement.executeUpdate(insertSQL);
            System.out.println(rowsInserted + " row(s) inserted.");
            // Update data
            String updateSQL = "UPDATE users SET email = 'ram@newdomain.com' WHERE name = 'Ram'";
            int rowsUpdated = statement.executeUpdate(updateSQL);
            System.out.println(rowsUpdated + " row(s) updated.");
            // Delete data
            String deleteSQL = "DELETE FROM users WHERE name = 'Shyam'";
            int rowsDeleted = statement.executeUpdate(deleteSQL);
            System.out.println(rowsDeleted + " row(s) deleted.");

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

▶ SQL Injection

- SQL Injection is a type of security vulnerability that occurs when an attacker is able to manipulate an application's SQL query by injecting malicious SQL code through user inputs.
- This process allows the attacker to bypass authentication, retrieve or modify sensitive data, delete records or even execute administrative operations on the database.
- The vulnerability arises when user-supplied data is directly inserted into SQL queries without proper validation or escaping, which makes it possible for the attacker to alter the structure of the query and execute unintended SQL commands.

Suppose there is a login form that accepts a username and password. If the user inputs are directly used in a SQL query like this:

```
SELECT * FROM users WHERE username = 'admin' AND password = ' ' OR '1' = '1';

//An attacker might manipulate the password as shown above. Since '1' = '1' is always true, the query would return all records, potentially bypassing authentication or exposing sensitive data.
```

Prepared Statements

- Prepared Statements are a robust and secure way to execute SQL queries in a database.
- They prevent SQL injection by treating user inputs as data rather than executable SQL code.
- Prepared statements separate SQL logic from the data being passed into the query, ensuring that any user-provided input is escaped and treated safely.
- They help to prevent the SQL Injection as user inputs are treated as data, not part of the query structure and also help to escape the special characters.
- Query logic and user input are clearly separated, making code easier to understand and maintain.
- Performance is optimized as the SQL query is parsed and compiled once and can be reused multiple times with different parameters, reducing query parsing overhead.
- Prepared statements support dynamic binding of values, making them flexible for a variety of queries.

Prepared Statements

```
import java.sql.*;
public class PreparedStatementExample {
    public static void main(String[] args) {
        try {
            // Establish the database connection
            Connection  conn = DriverManager.getConnection("dbc:mysql://localhost:3306/mydb", "root", "password");
            // SQL query with placeholders
            String sql = "SELECT * FROM users WHERE username = ? AND password = ?";
            // Create the PreparedStatement
            PreparedStatement  pstmt = conn.prepareStatement(sql);
            // Set the values for the placeholders
            pstmt.setString(1, "admin");  // Set first parameter
            pstmt.setString(2, "securepassword"); // Set second parameter
            // Execute the query
            ResultSet  rs = pstmt.executeQuery();
            // Process the results
            if (rs.next()) {
                System.out.println("Login successful! Welcome, " + rs.getString("username"));
            } else {
                System.out.println("Invalid username or password.");
            }
        } catch (SQLException e) {
            e.printStackTrace();
        } finally {
            // Close resources in reverse order of opening
            try {
                if (rs != null) rs.close();
                if (pstmt != null) pstmt.close();
                if (conn != null) conn.close();
            } catch (SQLException e) {
                e.printStackTrace();
            }
        }
    }
}
```